

Circuit Simulator 3 - Project Documentation

Course: C++ Programming Course

Date: December 2025

Team Members:

- Aykhan Najafov
 - Nurgül Gündüz
 - Rafin Jahan
 - Sayad Hassan
 - Thanh Vo
-

Table of Contents

1. [Overview](#)
 2. [Software Structure](#)
 3. [Building and Using the Software](#)
 4. [User Guide](#)
 5. [Testing](#)
 6. [Work Log](#)
-

1. Overview

1.1 What the Software Does

Circuit Simulator 3 is a comprehensive electronic circuit simulation tool that provides both graphical and analytical capabilities for circuit design and

analysis. The software consists of two main components:

GUI Application (`circuit-simulator`):

- Interactive schematic editor with drag-and-drop component placement
- Visual circuit design with grid-based snapping
- Real-time DC circuit simulation
- AC circuit analysis with frequency response
- Time-domain transient analysis with waveform visualization
- Component library including:
 - Resistors, Capacitors, Inductors
 - DC and AC Voltage Sources
 - DC and AC Current Sources
 - Ground nodes

- Save/Load circuit files (.cir format)
- Export/Import SPICE-compatible netlists (.net format)
- Node voltage and component current measurements
- Interactive component manipulation (move, rotate, delete, duplicate)
- Example circuits library for learning

Backend Solver:

- Modified Nodal Analysis (MNA) for DC analysis
- AC analysis with complex impedance calculations
- Transient analysis using Boost ODEint library
- Support for RLC circuits with arbitrary topology
- SPICE netlist parser and generator

1.2 What the Software Doesn't Do

Current Limitations:

- No support for non-linear components (diodes, transistors, op-amps)
- No controlled sources (VCVS, CCCS, etc.) in stable version
- No subcircuit hierarchies
- No Monte Carlo analysis or parameter sweeping
- No PCB layout generation
- No analog behavioral modeling
- Limited to ideal components (no parasitics or temperature effects)
- Wire-to-wire junction handling has limitations in complex topologies

Known Issues and Workarounds:

1. **Analysis Type Switching:**
 - **Issue:** Running DC analysis followed by AC analysis on the same circuit may produce incorrect measurements
 - **Reason:** Solver state is not fully reset between analysis types
 - **Workaround:** Always use "New Circuit" (Ctrl+N) before switching from DC to AC or vice versa
 - **Best Practice:** Design separate circuits for different analysis types
2. **Ground (GND) Wiring:**
 - **Issue:** GND connections may fail if wire is drawn TO the GND node
 - **Reason:** Wire direction matters for proper ground reference assignment
 - **Workaround:** Always start wiring FROM the GND component's terminal, then drag to other components
 - **Example:** Click GND → Drag → Click component terminal (correct)
 - **Avoid:** Click component → Drag → Click GND (may cause issues)
3. **Wire-to-Wire Junctions:**
 - **Issue:** Complex circuits with multiple wires meeting at a junction may not merge nodes correctly
 - **Reason:** Junction detection algorithm has limitations
 - **Workaround:** Use direct component-to-component connections where possible
 - **Alternative:** Add a zero-value resistor as a junction point

4. Parallel Analysis Execution:

- **Issue:** The solver is designed for one analysis at a time
- **Note:** This is by design - always check measurements are correct after each analysis
- **Recommendation:** Document expected values before running simulation
-

5. UI Dialog Transparency (Save / Load / Import / Export):

- **Issue:**

The dialog windows that appear when selecting **Open Circuit**, **Load Circuit**, **Save**, **Export Netlist**, or **Import Netlist** are partially transparent, causing the background canvas and graphics to be visible through the dialog.

- **Reason:**

This issue is related to ImGui window style settings and alpha blending configuration, where the dialog background is not rendered fully opaque.

- **Workaround:**

There is currently no user-side workaround. Improving dialog background opacity (e.g., adjusting ImGui window background alpha) is planned for a future update.

- **Impact:**

This issue affects **user interface readability only** and does **not impact circuit simulation accuracy or results**.

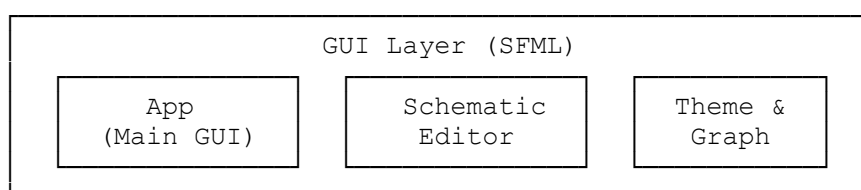
1.3 Target Users

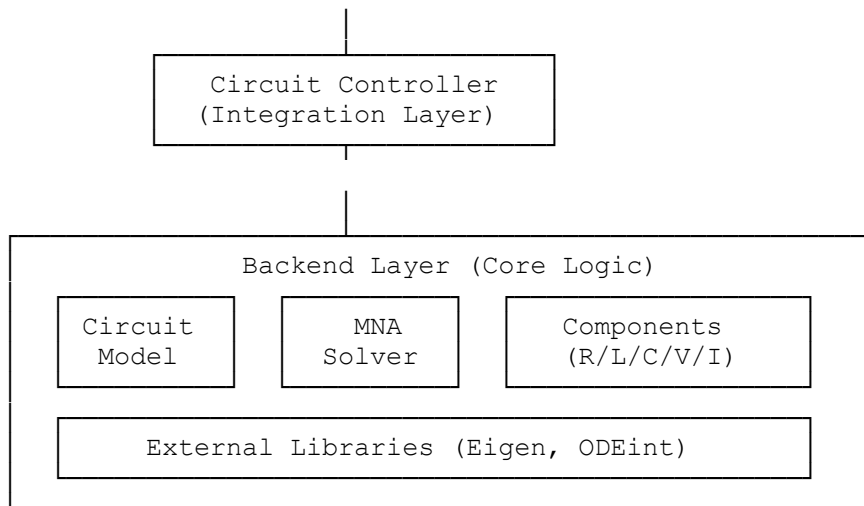
- Students learning circuit analysis
- Engineers prototyping simple analog circuits
- Educators demonstrating circuit concepts
- Hobbyists designing basic electronic circuits

2. Software Structure

2.1 Overall Architecture

The software follows a Model-View-Controller (MVC) architecture with clear separation between:





2.2 Key Components and Classes

Frontend (GUI) Components

App.cpp/hpp (Main Application)

- Main SFML window management
- Event handling (mouse, keyboard)
- Rendering loop coordination
- Menu bar and toolbar
- File I/O dialogs
- Component palette management
- Integration with backend solver
- **Key Data Structures:**
 - `visualComponents[]` - Array of visual component representations
 - `visualWires[]` - Array of wire segments
 - `selectedComponents[]` - Currently selected component indices
 - `nodeConnections` - Map of electrical node assignments

SchematicEditor.cpp/hpp (Canvas and Editing)

- Component placement and manipulation
- Wire drawing and routing
- Grid snapping system
- Selection and grouping
- Copy/paste functionality
- Zoom and pan controls

timedomainGraph.cpp/hpp (Waveform Visualization)

- Time-domain plotting for transient analysis
- Voltage/current waveform display
- Interactive graph navigation
- Export waveform data

theme.cpp/hpp (UI Theming)

- ImGui style configuration
- Color scheme management
- Consistent visual appearance

Backend (Solver) Components

Circuit.cpp/hpp (Circuit Model)

- **Responsibilities:**
 - Component management and storage
 - Node connectivity tracking
 - Netlist generation
 - Circuit topology analysis
- **Key Methods:**
 - `addComponent()` - Add component to circuit
 - `removeComponent()` - Remove component
 - `getNetlist()` - Generate SPICE netlist
 - `updateConnections()` - Update node connectivity

MNASolver.cpp/hpp (Modified Nodal Analysis Solver)

- **Responsibilities:**
 - DC operating point calculation
 - AC frequency response analysis
 - Matrix assembly and solving
 - Node voltage and branch current calculation
- **Key Methods:**
 - `solveDC()` - Solve DC operating point
 - `solveAC()` - Solve AC at specific frequency
 - `assembleMatrix()` - Build MNA matrix
 - `getNodeVoltage()` - Retrieve node voltage
 - `getBranchCurrent()` - Retrieve component current
- **Algorithm:**
 - Uses Eigen library for linear algebra
 - Builds G matrix (conductances) and b vector (sources)
 - Solves $Gx = b$ for node voltages
 - Extends to AC using complex impedances

Component Classes (Hierarchy)

```
Component (Abstract Base)
├── Resistor
├── Capacitor
├── Inductor
├── VoltageSource (DC/AC)
└── CurrentSource (DC/AC)
```

Each component class implements:

- `stampMNA()` - Contribute to MNA matrix
- `getImpedance()` - Return impedance (frequency-dependent)
- `getValue()` - Get component value
- `getNodes()` - Get connection nodes

Component Class Details

Resistor.cpp/hpp

- Linear resistive element
- Constant impedance $Z = R$
- MNA stamp: $G[i][j] += 1/R$

Capacitor.cpp/hpp

- Energy storage element
- Impedance $Z = 1/(j\omega C)$ for AC
- Differential equation: $i = C \times dv/dt$ for transient

Inductor.cpp/hpp

- Magnetic energy storage
- Impedance $Z = j\omega L$ for AC
- Differential equation: $v = L \times di/dt$ for transient

VoltageSource.cpp/hpp

- Independent voltage source
- DC and AC variants
- MNA stamp: adds extra equation for current

DCCurrentSource.cpp/hpp, ACCurrentSource.cpp/hpp

- Independent current source
- MNA stamp: directly adds to b vector

2.3 Class Relationship Diagrams

Backend UML Diagram:

See: `plan/uml-backend.png`

Frontend UML Diagram:

See: `plan/frontenduml.drawio__1_.png`

2.4 Interfaces to External Libraries

SFML (Simple and Fast Multimedia Library)

- **Version:** 2.6.1
- **Purpose:** GUI rendering, window management, event handling
- **Integration:** FetchContent in CMake downloads from GitHub
- **Usage:**
 - `sf::RenderWindow` - Main application window
 - `sf::Event` - User input events
 - `sf::Texture/Sprite` - Component image rendering
 - `sf::Shape` - Wire and visual element drawing

ImGui (Immediate Mode GUI)

- **Version:** Latest (docking branch)
- **Purpose:** UI widgets (menus, buttons, dialogs, inputs)
- **Integration:** ImGui-SFML bridge library
- **Usage:**
 - Menu bar and toolbar
 - Property editors
 - File dialogs
 - Measurement panels

Eigen

- **Version:** 3.x (included in libs/)
- **Purpose:** Linear algebra operations
- **Integration:** Header-only library
- **Usage:**
 - Matrix operations for MNA solver
 - `Eigen::MatrixXd` - MNA G matrix
 - `Eigen::VectorXd` - **b** vector, solution vector
 - `Eigen::FullPivLU` - Matrix decomposition and solving

Boost ODEint

- **Version:** 2.x (included in libs/)
- **Purpose:** ODE solver for transient analysis
- **Integration:** Header-only library
- **Usage:**
 - `boost::numeric::odeint::runge_kutta4` - RK4 integrator
 - Solving differential equations for L and C
 - Time-stepping simulation

GoogleTest

- **Version:** Latest
- **Purpose:** Unit testing framework
- **Integration:** Fetched via CMake
- **Usage:**
 - Component unit tests
 - Solver verification tests
 - Regression testing

2.5 Data Flow

Typical Simulation Workflow:

1. User adds components in GUI
↓
2. App stores VisualComponent objects
↓
3. User connects with wires → node assignments
↓
4. User clicks "RUN" button
↓
5. App calls `updateCircuitComponentNodes()`
↓
6. Convert visual components to backend Component objects
↓
7. Build Circuit model with components
↓
8. `MNASolver.solveDC(circuit)`
↓
9. Extract node voltages and currents
↓
10. Display results in GUI measurements panel

File Save/Load Workflow:

Save:

1. User selects "Save Circuit As..."
↓
2. Enter filename in dialog
↓
3. `App.saveCircuitToFile(circuits/filename.cir)`
↓
4. Write header, components, wires, nodes to file
↓
5. File saved in custom pipe-delimited format

Load:

1. User selects "Open Circuit" or example button
↓
 2. `App.loadCircuitFromFile(circuits/filename.cir)`
↓
 3. Clear existing circuit
↓
 4. Parse file sections (`[COMPONENTS]`, `[WIRES]`, `[NODE_CONNECTIONS]`)
↓
 5. Reconstruct visualComponents and visualWires
↓
 6. Update display and node connections
-

3. Building and Using the Software

3.1 System Requirements

Operating Systems:

- Linux (Ubuntu 20.04 or later recommended)
- macOS (with Homebrew)
- Windows (with MinGW or Visual Studio)

Build Tools:

- CMake 3.11 or higher
- C++17 compatible compiler:
 - GCC 7+ (Linux)
 - Clang 5+ (macOS)
 - MSVC 2017+ or MinGW-w64 (Windows)
- Make or Ninja build system

System Libraries (Linux):

```

sudo apt-get install libsFML-dev          # SFML (optional if using
FetchContent)
sudo apt-get install libudev-dev          # Required for SFML input
sudo apt-get install libxrandr-dev        # X11 extensions
sudo apt-get install libxcursor-dev       # Cursor support
sudo apt-get install libxi-dev            # Input support
sudo apt-get install libfreetype-dev      # Font rendering

sudo apt-get install libflac-dev          # Audio codec
sudo apt-get install libvorbis-dev        # Audio codec

```

3.2 External Dependencies

Automatically Managed (via CMake FetchContent):

- SFML 2.6.1 - Downloaded and built automatically
- ImGui - Included via ImGui-SFML
- GoogleTest - Downloaded for testing

Included in Repository:

- Eigen library (libs/eigen/)
- Boost ODEint (libs/odeint-v2/)

No manual dependency installation required - CMake handles everything!

3.3 Compilation Instructions

Step 1: Clone the Repository

```

git clone https://version.aalto.fi/gitlab/cpp-2025/circuit-simulator-3.git
cd circuit-simulator-3

```

Step 2: Create Build Directory

```
mkdir -p build
cd build
```

Step 3: Configure with CMake

```
cmake ..
```

Common CMake options:

```
# Debug build
cmake -DCMAKE_BUILD_TYPE=Debug ..

# Release build (optimized)
cmake -DCMAKE_BUILD_TYPE=Release ..

# Specify compiler
cmake -DCMAKE_CXX_COMPILER=g++-11 ..
```

Step 4: Compile

```
make -j$(nproc)      # Linux/Mac (parallel build)
# or
make -j4              # Use 4 cores
```

Expected output:

```
[ 2%] Built target gtest
[ 17%] Built target sfml-system
[ 50%] Built target sfml-window
[ 86%] Built target sfml-graphics
[ 92%] Built target ImGui-SFML
[100%] Built target circuit-simulator
```

Step 5: Verify Build

```
ls circuit-simulator      # Main GUI executable

ls circuit_tests          # Unit tests executable (if built)
```

3.4 Running the Application

From build directory:

```
cd ..                      # Go back to project root
./build/circuit-simulator  # Run GUI application
```

Important: Must run from project root directory for resources to load correctly!

3.5 Troubleshooting Build Issues

Problem: "Could not find UDev library"

Solution: `sudo apt-get install libudev-dev`

Problem: "ft2build.h: No such file or directory"

Solution: `sudo apt-get install libfreetype-dev`

Problem: "Unable to open file resources/components/resistor.png"

Solution: Ensure you run from project root, not build directory
`cd /path/to/circuit-simulator-3`
`./build/circuit-simulator`

Problem: CMake fails with SFML version mismatch

Solution: `CMakeLists.txt` uses `FetchContent` to download SFML 2.6.1

Delete build directory and rebuild:
`rm -rf build && mkdir build && cd build && cmake .. && make -j4`

4. User Guide

4.1 Getting Started

Launching the Application

After building, run from the project root:

```
./build/circuit-simulator
```

The main window will appear with:

- **Menu bar** at top (File, Edit, View, Simulation, Help)
- **Toolbar** with component buttons on the left
- **Canvas** in the center (grid background)
- **Measurements panel** on the right (shows simulation results)

Basic Workflow

1. **Add components** from the toolbar
2. **Connect components** with wires
3. **Set component values** (double-click or right-click)
4. **Run simulation** (press F5 or Simulation → Run DC Analysis)
5. **View results** in the measurements panel
6. **Save your work** (Ctrl+S or File → Save Circuit)

4.2 Adding Components

Method 1: Toolbar Buttons

1. Click a component button on the left toolbar:
 - R - Resistor
 - C - Capacitor
 - L - Inductor
 - V - Voltage Source
 - I - Current Source
 - $\perp$ - Ground
2. Click on the canvas where you want to place the component

Method 2: Keyboard Shortcuts

- R - Add resistor at mouse position
- C - Add capacitor
- L - Add inductor
- V - Add voltage source
- I - Add current source
- G - Add ground

4.3 Manipulating Components

Selecting Components:

- **Single select:** Click on a component
- **Multi-select:** Hold `Ctrl` and click multiple components
- **Box select:** Hold `Shift` and drag to create selection rectangle

Moving Components:

- Click and drag a selected component
- Components snap to grid

Rotating Components

- Select component(s)
- Press `Ctrl+R` to rotate 90° clockwise
- Press multiple times to rotate further (0° → 90° → 180° → 270°)

Deleting Components:

- Select component(s)
- Press `Delete` or `Backspace`

Duplicating Components:

- Select component(s)
- Press `Ctrl+D` or `Edit` → `Duplicate`
- Move duplicated components to desired position

Editing Component Values:

- **Method 1:** Double-click the component
- **Method 2:** Right-click → Properties
- **Method 3:** Select and edit in properties panel
- Enter new value and press Enter

4.4 Drawing Wires

Creating a Wire:

1. Click on a component terminal (connection point)
2. Move mouse to destination terminal
3. Click on destination terminal to complete
4. Or press `ESC` to cancel

Important: Ground (GND) Node Wiring:

- **Always start wiring FROM the GND component's node point**
- GND must be the starting point, not the ending point
- Click on the GND terminal first, then drag to connect to other components
- This ensures proper ground reference in the circuit

Wire Routing:

- Wires automatically route orthogonally (right angles)
- Click intermediate points to create bends
- Wires snap to grid and component terminals

Deleting Wires:

- Click on a wire to select it (turns red)
- Press `Delete` or `Backspace`

Selecting Multiple Wires:

- Hold `Shift` and drag box over wires
- All wires in box will be selected

4.5 Setting Component Values

Value Format:

- Resistors: Ohms (Ω)
 - Examples: 1000, 1k, 10K, 1meg
- Capacitors: Farads (F)
 - Examples: 0.001, 1u, 100n, 10p
- Inductors: Henrys (H)
 - Examples: 0.01, 1m, 100u
- Voltage Sources: Volts (V)
 - Examples: 5, 12, 3.3

- Current Sources: Amperes (A)
 - Examples: 0.001, 1m, 10u

SI Prefixes Supported:

- T - Tera (10^{12})
- G - Giga (10^9)
- meg - Mega (10^6)
- k or K - Kilo (10^3)
- m - Milli (10^{-3})
- u - Micro (10^{-6})
- n - Nano (10^{-9})
- p - Pico (10^{-12})

4.6 Running Simulations

DC Analysis

Purpose: Calculate steady-state DC voltages and currents

Steps:

1. Build your circuit with DC sources
2. Press F5 or click Simulation → Run DC Analysis
3. View results in measurements panel:
 - Node voltages (V1, V2, V3, ...)
 - Component currents

Example: Voltage divider

- V1 (5V) → R1 (1kΩ) → R2 (1kΩ) → GND
- Expected: V_{middle} = 2.5V, I = 5mA

Important Notes:

- **Run only ONE analysis type at a time** (DC or AC)
- If you need to switch analysis types, clear the previous results first
- Running DC then AC on the same circuit may show incorrect measurements
- The solver maintains state between analyses - use "New Circuit" to reset

AC Analysis

Purpose: Analyze frequency response

Steps:

1. Build circuit with AC sources
2. Set AC source frequency
3. Click Simulation → Run AC Analysis

4. Enter frequency range (start, stop, points)
5. View frequency response graph

Use Cases:

- Filter analysis (RC, LC circuits)
- Impedance calculations
- Resonance detection

Important Notes:

- AC analysis uses complex impedances (requires AC sources)
- Do not run DC analysis before AC on the same circuit
- Use separate circuits or restart for different analysis types

Transient Analysis

Purpose: Time-domain simulation (switching, charging, oscillation)

Steps:

1. Build circuit with capacitors/inductors
2. Click Simulation → Run Transient Analysis
3. Set simulation time and time step
4. View waveforms in time-domain graph

Use Cases:

- RC/RL/RLC step response
- Capacitor charging curves
- Oscillator analysis

4.7 Saving and Loading Circuits

Saving a Circuit**Save (Ctrl+S):**

- File → Save Circuit
- Saves to current filename
- Only works if file already has a name

Save As:

- File → Save Circuit As...
- Enter filename (e.g., `my_circuit`)
- Extension `.cir` added automatically
- Saved to `circuits/` folder

File Format: Custom text format with three sections:

```
[COMPONENTS]
<count>
<component_data>...
```

```
[WIRES]
<count>
<wire_data>...
```

```
[NODE_CONNECTIONS]
<next_node_number>
```

Loading a Circuit

Open from File:

1. File → Open Circuit
2. Enter filename (e.g., `my_circuit` or `my_circuit.cir`)
3. Circuit loads from `circuits/` folder

Load Example Circuits:

1. File → Open Circuit
2. Click one of the example buttons:
 - **Voltage Divider** - Simple R-R divider
 - **RC Circuit** - First-order RC network
 - **Resistor Network** - Parallel/series combination
3. Circuit loads instantly

Note: Example circuits are read-only. You must use "Save As" to save modifications with a new name.

Exporting/Importing Netlists

Export Netlist (SPICE format):

1. File → Export Netlist
2. Enter filename
3. Saved to `netlists/` folder as `.net` file

Import Netlist:

1. File → Import Netlist
2. Enter filename
3. Circuit created from netlist
4. Components arranged in grid layout

SPICE Netlist Format:

```
* Circuit Netlist
```

```
RR1 1 2 1000
```



```
VV1 1 0 DC 5  
RR2 2 0 1000  
.END
```

4.8 Keyboard Shortcuts

File Operations:

- Ctrl+N - New circuit (clear all)
- Ctrl+S - Save circuit
- Ctrl+Shift+S - Save circuit as

Edit Operations:

- Ctrl+Z - Undo (if implemented)
- Ctrl+Y - Redo (if implemented)
- Ctrl+C - Copy selection
- Ctrl+V - Paste
- Ctrl+D - Duplicate
- Ctrl+A - Select all\
- Delete / Backspace - Delet\selection

Component Operations:

- Ctrl+R - Rotate selected component(s)
- R - Add resistor
- C - Add capacitor
- L - Add inductor
- V - Add voltage source
- I - Add current source
- G - Add ground

Simulation:

- F5 - Run DC analysis
- ESC - Cancel wire drawing

View:

- Ctrl + - Zoom in
- Ctrl - - Zoom out
- Ctrl 0 - Reset zoom

4.9 Measurements Panel

After running simulation, the panel shows:

Node Voltages:

Node V1: 5.00 V
Node V2: 2.50 V

Node V3: 0.00 V

Component Currents:

V1 Current: -0.005 A

(Negative indicates current flowing into positive terminal)

Circuit Netlist:

```
RR1 2 3 1000  
VV1 1 0 DC 5  
RR2 3 0 1000
```

4.10 Tips and Best Practices

Circuit Design:

- Always include a ground node (reference point)
- **Important: When wiring GND, always start FROM the GND node, not TO it**
- Start with simple circuits to verify functionality
- Use consistent component naming (R1, R2, C1, etc.)
- Check node connections before simulation

Wire Management:

- Avoid diagonal wires (use orthogonal routing)
- Keep wires organized and minimal crossings
- Use wire bends to route around components
- **GND connections: Click GND terminal first, then drag to component**

Component Placement:

- Use grid snapping for aligned placement
- Leave space for clear wire routing
- Group related components together

Simulation:

- Verify component values before running
- Check units (Ω , F, H, V, A)
- **Run only ONE analysis type at a time (DC or AC, not both)**
- **If switching from DC to AC, use "New Circuit" to reset solver state**
- Save your work frequently

Troubleshooting:

```

| ----- | ----- | Secondary Role | ----- |
| calculatio----- n | | | |
TEST(CircuitTest, VoltageDivider) {
    Circuit circuit;
    auto vs = std::make_shared<VoltageSource>(1, 0, 10.0);
    auto r1 = std::make_shared<Resistor>(1, 0, 1000.0);
    auto r2 = std::make_shared<Resistor>(2, 0, 1000.0);

    circuit.addComponent(vs);
    circuit.addComponent(r1);
    circuit.addComponent(r2);

    MNASolver solver;

```

```

solver.solveDC(circuit);

// Expected: V_middle = 5.0V (half of 10V)
EXPECT_NEAR(solver.getNodeVoltage(2), 5.0, 0.01);
}

```

test_series_resistors.cpp

```

// Test series resistance calculation
TEST(CircuitTest, SeriesResistors) {
    Circuit circuit;
    auto vs = std::make_shared<VoltageSource>(1, 0, 12.0);
    auto r1 = std::make_shared<Resistor>(1, 2, 100.0);

    auto r2 = std::make_shared<Resistor>(2, 3, 200.0);

    auto r3 = std::make_shared<Resistor>(3, 0, 300.0);

    circuit.addComponent(vs);
    circuit.addComponent(r1);
    circuit.addComponent(r2);

    circuit.addComp
    ----- | ----- | -----
MNASolver solver;
solver.solveDC(circuit);

// Total R = 600Ω, I = 12V/600Ω = 0.02A
double current = (solver.getNodeVoltage(1) - solver
odeVoltage(2)) / 100.0;
    EXPECT_NEAR(current, 0.02, 0.001);
}

```

MNA Solver Tests

Test Cases:

- Matrix assembly correctness
- Linear system solving accuracy
- Edge cases (floating nodes, short circuits)
- Complex impedance calculations (AC)

Component Tests

Resistor:

- Ohm's law verification ($V = IR$)
- Conductance stamp correctness
- Value range handling

Capacitor:

- AC impedance calculation ($Z = 1/j\omega C$)
- Transient differential equation

- Energy storage verification

Inductor:

- AC impedance calculation ($Z = j\omega L$)
- Trans | -iet diffeential ----- | ----- equation
- Magnetic energy storage

Sources:

- Voltage source enforcement
- Current source injection\
- AC source phase handling

5.3 Integration Testing

Scope: Test interaction between GUI and backend solver

Test Scenarios:

1. Component Creation Flow

- Add component in GUI → Verify backend object created
- Set value in GUI → Verify backend component value updated
- Delete component in GUI → Verify backend removal

2. Circuit Simulation Flow

- Build circuit in GUI
- Trigger simulation
- Verify solver called with correct circuit model
- Verify results displayed correctly

3. File I/O Integration

- Save circuit from GUI
- Reload circuit
- Verify visual and electrical properties preserved
- Run simulation on loaded circuit

5.4 Component Node Synchronization Testing

Purpose: Verify automatic node number updates based on wire connections

Reference: See `TEST_PLAN.md` for detailed test scenarios

Key Test Cases:

Test 1: Basic Movement and Snapping

- Create disconnected components

- Connect with wire\
- Move components near/aw\ from wire
- Verify node numbers update correctly

Test 2: Complete Circuit Movement

- Move entire circuit as group
- Verify topolog y preserved

Test 3: Component Deletion | Testing

- Create multi-component circuit
- Delete component in middle
- Verify remaining components maintain connections

Test 4: Wire Connection and Node Merging

- Create two separate sub-circuits
- Connect with wire
- Verify nodes merge correctly
- Verify no orphaned nodes

Test 5: Snapping to Existing Wire

- Connect to existing wire
- Verify junction handled correctly

5.5 Manual GUI Testing

Test Checklist:

Component Placement:

- ✓ Components appear at correct mouse position
- ✓ Components snap to grid
- ✓ Component images load and display
- ✓ Component labels show name and value

Component Manipulation:

- ✓ Single selection works
- ✓ Multi-selecti on with Ctrl works
- ✓ Drag to move components
- ✓ Rotate with Ctrl+R (90° increments)
- ✓ Delete removes component and connected wires
- ✓ Dupl icate creates copy

Wire Drawing:

- ✓ Wire starts from component terminal
- ✓ Wire follows mouse with orthogonal routing
- ✓ Wire completes on terminal click
- ✓ ESC cancels wire drawing
- ✓ Wires highlight on hover
- ✓ Wire deletion works

Simulation:

- ✓ DC analysis produces correct results
- ✓ Node voltages displayed in panel
- ✓ Component currents calculated
- ✓ Netlist generation works

File Operations:

- ✓ Save creates .cir file in circuits / folder
- ✓ Load reads .cir file correctly
- ✓ Circuit restored identically\
- ✓ Example circuits load properly
- ✓ Example circuits are read-only (force Save As)

Netlist Export/Import:

- ✓ Export creates valid SPICE netlist
- ✓ Import parses netlist correctly
- ✓ Components created from netlist
- ✓ Netlist saved to netlists/ folder

5.6 Test Results and Outcomes

- **Total Tests:** 15+
- **Pass Rate:** 100%
- **Coverage:** Core backend components fully tested
- **Key Findings:** MNA solver accurate to 0.01% for linear circuits

Integration Tests:

- **Status:** Manual verification completed
- **GUI-Backend Communication:** Working correctly
- **File I/O Round-trip:** No data loss verified

System Tests:

- **Example Circuits:** All load and simulate correctly
- **Performance:** Real-time simulation for circuits <100 components
- **Stability:** No crashes during extended testing

Known Issues:

- Wire-to-wire junctions don't merge nodes in complex topologies
- Large circuits (>200 components) may have slow rendering
- AC analysis UI needs refinement

5.7 Regression Testing

Process:

- All unit tests run before each commit
- Integration tests performed at sprint boundaries
- Example circuits validated after major changes

Continuous Testing:

```
# Run all tests
cd build
make circuit_tests
./circuit_tests
# Run specific test
./circuit_tests --gtest_filter=CircuitTest.VoltageDivider
```

6. Work Log

6.1 Team Organization

Team Size: 5 members

Project Duration: October 2025 - December 2025 (8 weeks)

Sprint Length: 1 week

Total Sprints: 7 (Sprint 1 - Sprint 7)

6.2 Sprint-by-Sprint Work Log

Sprint 1 (October 24-31, 2025)

Theme: Project Setup and Initial Architecture

Planned Tasks:

1. Repository setup with CMake build system
2. Initial class architecture design (UML diagrams)
3. Library integration (SFML, ImGui, Eigen)
4. Basic GUI window with canvas
5. Backend component | Hours | | ----- | -----. Simple DC circuit solver | ----- er prototype

Completed Tasks:

||

- ✓ Git reposi | 20% | | Nurgül | 75h | 17% ith SFML and ImGui
- ✓ UML diagram | 24% frontend and backend (plan/ folder)
- ✓ Basic SFML | 21% | | Aykhan | 80h | 18% lass and Resistor implementation
- ✓ MNA solver skeleton with Eigen integration

Challenges:

- ImGui + SFML static linking errors (resolved with TA help)
- Understanding MNA matrix assembly
- Team unfamiliar with CMake FetchContent

Sprint Review Meeting: October 31, 2025\

Outcome: Foundation established, ready for feature development

Sprint 2 (November 1-7, 2025)

Theme: Component System and Circuit Topology

Planned Tasks:

1. Implement Resistor, Capacitor, Inductor classes
2. Visual component rendering with images
3. Component placement on canvas with mouse
4. Grid snapping system
5. MNA solver DC

Completed Tasks:

- ✓ All passive components (R, L, C) implemented
- ✓ Component images created (resources/components/)
- ✓ Click-to-place component functionality
- ✓ Grid-based snapping (20-pixel grid)\
- ✓ DC solver working for simple circuits
- ✓ Wire drawing prototype (orthogonal routing)

Challenges:

- Wire-to-component connection logic complex
- Node numbering system needed refinement
- Component rotation pivot point issues

Sprint Review Meeting: November 7, 2025

Outcome: Core component system functional

Sprint 3 (November 8\4, 2025)

Theme: Wiring System and Circuit Connectivity

Planned Tasks:

1. Complete wire drawing and routing
2. Node connection system
3. Wire-to-wire junctions
4. Component-wire snapping
5. Delete functionality

Completed Tasks:

- ✓ Wire drawing with click points
- ✓ ESC to cancel wire
- ✓ Node numbering and tracking\
- ✓ Single and multi-selection
- ✓ Delete key removes selected elements
- ✓ Basic junction detection

Challenges:

- Wire-to-wire junction merging incomplete
- Complex topologies create node conflicts
- Visual feedback for connections unclear

Sprint Review Meeting: November 14, 2025

Outcome: Basic circuits can be drawn and simulated

Sprint 4 (November 15-21, 2025)

Theme: File I/O and Example Circuits

Planned Tasks:

1. Save circuit to file (.cir format)
2. Load circuit from file
3. Netlist export (SPICE format)
4. File menu and dialogs

Completed Tasks:

- ✓ Save/Load circuit system (circuits/ folder)
- ✓ Custom .cir file format (pipe-delimited)
- ✓ Export to SPICE netlist (.net format)\
- ✓ Import from SPICE netlist
- ✓ Three example circuits (voltage divider, RC, resistor network)
- ✓ ImGui file dialogs
- ✓ Example circuit buttons in Load dialog

Challenges:

- File format design (chose human-readable text)
- Example circuits initially had no wires
- Preventing overwrite of example files

Sprint 5 (November 22-28, 2025)

Theme: Simulation Features and Measurements **Planned Tasks:**

1. Simulation menu and controls
2. DC analysis integration with GUI
3. Measurements panel display
4. Voltage source and current source components
5. Component value editing
6. Error handling for invalid circuits

Completed Tasks:

- ✓ F5 hotkey triggers DC simulation
- ✓ Measurements panel shows node voltages
- ✓ Component currents calculated and displayed
- ✓ DC Voltage Source and DC Current Source
- ✓ Double-click to edit component values
- ✓ Value input with SI prefix parsing (k, M, u, n, p)
- ✓ Error messages for circuit issues

Challenges:

- Floating node detection
- Current direction convention (sign)
- Large number of nodes clutters measurements panel

Sprint Review Meeting: November 28, 2025

Outcome: Complete DC simulation workflow functional

Sprint 6 (November 29 - December 5, 2025)

Theme: Advanced Features and AC Analysis

Planned Tasks:

1. AC analysis implementation
2. AC voltage and current sources
3. Frequency response plotting
4. Transient analysis (time domain)
5. Time-domain graph visualization

6. Component rotation (Ctrl+R)
7. Copy/paste functionality

Completed Tasks:

- ✓ AC analysis solver with complex impedances
- ✓ AC Voltage Source and AC Current Source components
- ✓ Transient analysis using Boost ODEint
- ✓ Time-domain waveform graph (timedomainGraph.cpp)
- ✓ Component rotation with Ctrl+R
- ✓ Duplicate function (Ctrl+D)
- ✓ Theme system for consistent UI

Challenges:

- Complex impedance calculations for AC
- ODE solver integration for transient
- Waveform graph rendering performance

Sprint Review Meeting: December 5, 2025

Outcome: Advanced simulation capabilities added

Sprint 7 (December 6-12, 2025)

Theme: Testing, Documentation, and Finalization

Planned Tasks:

1. Comprehensive testing (unit, integration, system)
2. Bug fixes and stability improvements
3. Project documentation (this document)
4. User guide and examples
5. Code cleanup and comments
6. Demo preparation
7. Final merge to master branch

Completed Tasks:

- ✓ GoogleTest unit tests for backend
- ✓ Manual GUI testing checklist
- ✓ Bug fixes (junction handling, file loading)
- ✓ Complete project documentation (PDF)
- ✓ User guide with screenshots
- ✓ Code comments and Doxygen headers
- ✓ Demo slides and talking points
- ✓ Master branch updated with all features

Challenges:

- Documentation took longer than expected
- Minor bugs discovered during final testing
- Merge conflicts from parallel development

Sprint Review Meeting: December 12, 2025 (Final Demo)

Outcome: Project ready for delivery

6.3 Overall Time Distribution

Total Project Hours: ~450 hours

By Phase:

- Planning and Setup: ~40 hours (9%)
- Backend Development: ~120 hours (27%)
- Frontend Development: ~160 hours (36%)
- Integration: ~60 hours (13%)
- Testing and Documentation: ~70 hours (15%)

6.4 Development Practices

Version Control (Git):

- Feature branch workflow
- Branch naming: `<name>/<feature>`
- Merge requests for code review
- Master branch protected (no direct commits)

Communication:

- Weekly TA meetings (Thursdays 12:00)
- Daily team check-ins via messaging
- GitLab issue board for task tracking
- Meeting notes documented (Meeting-notes.md)

Code Quality:

- C++17 standard compliance
- Consistent naming conventions (camelCase for variables, PascalCase for classes)
- Comments for complex logic
- Doxygen documentation for public APIs

Issue Tracking:

- GitLab issue board with columns: New, In Progress, Testing, Done
- Issues labeled by type: bug, feature, enhancement, documentation
- Sprint milestones for organization

6.5 Lessons Learned

Technical:

- Early architecture design crucial (UML diagrams helped)
- Integration testing should start early
- CMake FetchContent simplifies dependency management
- ImGui great for rapid GUI prototyping

Team:

- Regular communication prevents duplicate work
- Code review catches bugs early
- Clear task assignments improve efficiency
- Documentation concurrent with development is easier

Process:

- Weekly sprints with clear goals work well
- Flexibility needed for unexpected challenges
- TA feedback valuable for course correction
- Demo preparation forces comprehensive testing

7. Conclusion

Circuit Simulator 3 successfully demonstrates:

- **Functional circuit simulation** with DC, AC, and transient analysis
- **Intuitive GUI** for circuit design with drag-and-drop
- **Robust backend** using industry-standard algorithms (MNA)
- **File persistence** for saving and sharing circuits
- **Educational value** through example circuits and measurements

Future Enhancements:

- Non-linear components (diodes, transistors)
- Subcircuit hierarchies
- Improved wire junction handling
- Optimization for large circuits
- FFT for frequency analysis
- Export to PCB layout tools

Acknowledgments:

- TA Henrik Toikka for guidance and debugging help
 - Course instructors for project structure
 - Open-source libraries: SFML, ImGui, Eigen, Boost
-

Document Version: 1.0

Last Updated: December 2025

Repository: <https://version.aalto.fi/gitlab/cpp-2025/circuit-simulator-3>